

MODULE 16 · DOMAIN 5 - CONTEXT MANAGEMENT & RELIABILITY

Long Sessions and Codebase Exploration

Why re-reading a file does not refresh it, and how to explore 800 files without loading 800 files.

DOMAIN 5 of 5 EXAM WEIGHT 15% MODULE 16 of 20

Two problems, often confused

	VOLUME	STALENESS
Cause	Too much has entered the window	The world changed since it entered
Symptom	Vaguer answers, forgotten constraints	Confident answers based on a version that no longer exists
Fix	Reduce what enters; summarise; delegate	Refresh, or start again with what you know now
Time	Grows with the length of the session	Grows with the gap since you last looked

ON THE EXAM

A big window makes volume worse and staleness no better. They need different answers.

A conversation is **append-only**. You cannot overwrite a fact in it. You can only add a newer one beside it.

Why re-reading is not refreshing

- ◆ The old contents are still there, sitting earlier in the same conversation.
- ◆ Nothing timestamps them as superseded. Both reads look equally authoritative.
- ◆ The agent may reason from either, and it will not tell you which it used.
- ◆ This is arguably worse than not re-reading, because you now believe the context is current.

ON THE EXAM

Re-reading adds a version. It never removes one. That is the whole reason this is a hard problem.

How to actually refresh

- ◆ **Say what changed and that the earlier read is superseded.** Make the newness explicit, in words.
- ◆ **Name the files.** Three specific paths beats "some things have changed".
- ◆ **Or clear the tool results** so the stale reads are gone rather than merely outranked.
- ◆ **Or start fresh with a summary** when a lot has moved. Cleanest, and it costs you the detail.

ON THE EXAM

Naming exactly which files changed is cheap and effective. Telling it to re-read everything is neither.

Refresh in place, or start again?

REFRESH IN PLACE

A few things moved

- You know exactly what changed
- The rest of the context is still valuable
- State plainly that the earlier read is stale
- Cheap, and it keeps the reasoning chain

START AGAIN

A lot moved, or you cannot tell

- The gap was long, or the change was broad
- Too many stale reads to enumerate
- Inject a summary of decisions and current state
- Costs the detail; buys certainty

THE TIEBREAKER

Can you list what changed? If yes, refresh. If you are not sure what changed, that uncertainty is the answer.

The handoff summary, refined

- ◆ **Decisions and why**, so the new session does not relitigate them.
- ◆ **Current state, described as current**, not as it was when you started.
- ◆ **What is still open, and what was ruled out**. Otherwise it suggests the thing you rejected.
- ◆ **Not the tool results**. Those are the stale part you are escaping.
- ◆ **Not the transcript**. Pasting it back rebuilds the problem by hand.

ON THE EXAM

The two exclusions matter as much as the inclusions. A summary that carries the stale reads is not a fresh start.

A coding agent's context is mostly **files it read**. So the skill is reading fewer of them.

The exploration pattern

01

Locate

Glob for paths, Grep for content. Neither loads a file.

02

Narrow

Filter by type, tighten the pattern, count matches first.

03

Read

Open only the files the search actually implicated.

04

Follow

Imports, callers, tests. Move outward from what you read.

ON THE EXAM

Locate, narrow, read, follow. Every step before "read" exists to make the read smaller.

Search first, or read everything?

SEARCH FIRST

Find, then open

- Paths and matched lines are cheap
- Context grows with relevance, not repo size
- Works identically on 80 files or 8,000
- The agent builds understanding incrementally

READ EVERYTHING

Load, then reason

- Every file enters context whether relevant or not
- Context grows with the repository
- Fine on a toy project, fatal on a real one
- Degradation arrives within a few questions

THE TIEBREAKER

Loading every file to answer a question about two of them is the most expensive way to be thorough.

What each tool costs you

TOOL	WHAT IT DOES	CONTEXT COST
<code>Glob</code>	Matches file paths by name pattern	Paths only. Cheap.
<code>Grep</code>	Searches file contents	Paths, or matched lines. Cheap.
<code>Read</code>	Loads a file's contents	The whole file. Expensive.
<code>Edit</code>	Changes specific text in place	Cheaper than reading and rewriting.

ON THE EXAM

Glob is for names. Grep is for contents. Confusing the two is a common wrong answer.

Getting more out of Grep

- ◆ **Three output modes.** File paths only, matching lines with numbers, or a count per file.
- ◆ **Count first when you do not know the scale.** Forty matches is a plan; four hundred is a rethink.
- ◆ **Filter by file type or glob pattern** to cut the search down before it runs.
- ◆ **Ask for a few lines of surrounding context** rather than opening the file, when you only need the shape.

ON THE EXAM

Counting before reading is the cheapest thing in this module and almost nobody does it.

Following the graph

- ◆ **Imports tell you what this file depends on.** Follow them when you need the definitions.
- ◆ **Callers tell you what depends on this.** Grep for the function name to find them.
- ◆ **Tests tell you what it is supposed to do,** often more clearly than the code does.
- ◆ **Follow deliberately, one hop at a time.** Every hop is a file, and every file is context.

ON THE EXAM

"Read the file and follow its imports" is a bounded strategy. "Read the module" is not.

When to hand exploration to a subagent

- ◆ **When the exploration is genuinely broad** and will touch many files to produce a short answer.
- ◆ **When the parent's context is already degrading** and you need it to stay sharp.
- ◆ **Not for a targeted search.** A subagent costs a round trip and a fresh context load.
- ◆ **Claude Code ships an Explore subagent** for exactly this, and it is the cheap option for read-heavy work.

ON THE EXAM

Reach for a subagent when a targeted Grep genuinely is not enough. It is slower, and it is for real breadth.

Forking to explore two approaches

- ◆ **Fork before an experiment**, so the thread you are on survives it.
- ◆ **Both branches stay resumable**, with separate session IDs and unchanged histories.
- ◆ **A fork inherits the parent's context**, so it does nothing for context pressure.
- ◆ **Compare the outcomes, then continue from whichever branch won.**

ON THE EXAM

Fork is for branching a decision. Subagent is for isolating context. They look similar and solve different problems.

Reading the stem: session and exploration questions

WHAT THE STEM SAYS	WHAT IT WANTS
"a colleague merged while the agent was idle"	Fresh session with a summary, or a named refresh
"we asked it to re-read the file and it still got it wrong"	Append-only. Both versions are present.
"which files changed" is knowable	Name them. Cheap and effective.
"800 files, most questions touch two"	Search first, then read what the search implicates
"find files called" versus "find files containing"	Glob versus Grep
"the exploration output is huge, the answer is short"	A subagent, or a forked skill

ON THE EXAM

Staleness questions turn on what happened in the gap. Exploration questions turn on how much gets loaded.

Module 16 in five lines

- ◆ Volume and staleness are different problems with different fixes.
- ◆ Conversations are append-only. Re-reading adds a version; it never removes one.
- ◆ Name what changed when you can. Start fresh when you cannot.
- ◆ Locate, narrow, read, follow. Every step before the read makes the read smaller.
- ◆ Glob is for names, Grep is for contents, and neither loads a file.

ON THE EXAM

Five scenarios next. One of them turns entirely on the append-only fact.

An engineer resumes a session after lunch. Three files the agent had read have since been modified by a colleague.

The engineer asks the agent to re-read those three files before continuing. It does. It then produces code that uses a function signature from the pre-lunch version.

Nothing was truncated and no error occurred.

What is happening?

- ☐ A The re-read failed silently and returned cached contents
- ☐ B Both versions are now in the conversation, and nothing marks the older one as superseded
- ☐ C The agent cannot read files that changed during a session
- ☐ D The context window filled and the newer read was dropped

◆ PAUSE AND DECIDE

What is happening?

- A

 Silent cache failure
Invents a caching layer. The read succeeded; the stem says it re-read them.
- B

 Both versions present
A conversation is append-only. The new read sits after the old one, and neither is marked current.
- C

 Cannot read changed files
Invented limitation. Reading a file has nothing to do with when it changed.
- D

 Window filled
That would raise an error, and the stem rules out both truncation and errors.

THE SIGNAL

The engineer did the intuitive thing and it made the context worse. That is why this scenario exists.

An agent answers questions about a repository of roughly 800 files. Its current approach is to read every file in the relevant top-level directory before answering.

Directories range from 12 to 300 files. Answers on the small directories are good; on the large ones they are noticeably worse and slower.

Analysis shows most questions concern one or two files.

What is the most effective change?

- ☐ A Read the directory in batches, summarising each batch before moving on
- ☐ B Grep for the relevant symbols, read only the files that match, then follow imports as needed
- ☐ C Build an index of the repository and load it at the start of each session
- ☐ D Read files in order of modification time, most recent first

◆ PAUSE AND DECIDE

What is the most effective change?

A Batch and summarise

Every file still enters context before being summarised, and the summaries are tokens on top.

C Load an index

A standing context cost on every session, to answer questions that a live search answers for nothing.

B Search, read, follow

Context grows with relevance rather than directory size. Most questions touch two files, so read two.

D Most recent first

Reorders the reading without reducing it. Recency and relevance are not the same thing.

THE SIGNAL

"Good on small directories, worse on large ones" tells you the volume is the variable. The strategy scales with the wrong thing.

An engineer needs to find every place a deprecated function called `calculateTax` is still being called, across a large codebase.

They also need to find all the configuration files, which follow the naming convention `config.*.json`.

They are deciding which tools the agent should use.

Which combination is correct?

- | | | | |
|----------------------------|--|----------------------------|--|
| ☐ A | Glob for <code>calculateTax</code> , Grep for <code>config.*.json</code> | ☐ B | Grep for <code>calculateTax</code> , Glob for <code>config.*.json</code> |
| ☐ C | Grep for both, since Grep can match filenames too | ☐ D | Read the directory listing, then Read each candidate file |

◆ PAUSE AND DECIDE

Which combination is correct?

- A

 Glob then Grep
Exactly reversed. Glob does not look inside files, so it cannot find a function call.
- B

 Grep then Glob
Function calls are content, so Grep. Filenames are paths, so Glob. Each tool for its purpose.
- C

 Grep for both
Technically possible for the filenames, and Glob is the purpose-built tool and cheaper for path matching.
- D

 List and read
Loads every candidate file to answer a question that two searches answer without loading anything.

THE SIGNAL

Content goes to Grep. Paths go to Glob. Option A is the same answer reversed, which is how this is usually tested.

A developer is deep in a complex refactor with an agent. They want to try a substantially different approach, but do not want to lose the current thread if it does not work out.

Separately, the agent's context is around 70% full and its recent suggestions have become less precise.

A colleague suggests running the experiment as a subagent, since that keeps context isolated.

What should they do?

- ☐ A Run the experiment as a subagent, as suggested
- ☐ B Fork the session for the experiment, and separately address the context pressure
- ☐ C Start a fresh session for the experiment with no context
- ☐ D Continue in the same session and revert the changes if the experiment fails

◆ PAUSE AND DECIDE

What should they do?

A Subagent

Subagents inherit nothing. An experiment continuing a deep refactor needs the refactor's context.

C Fresh session

Throws away the refactor context the experiment depends on. Right mechanism, wrong moment.

B Fork, then handle context separately

Fork branches a decision while keeping the thread. The context pressure is a second problem needing its own fix.

D Continue and revert

Reverting files does not revert the conversation. The abandoned approach stays in context, confusing later turns.

THE SIGNAL

Two problems in one stem, and the colleague's suggestion conflates them. Fork solves one; it does not solve the other.

An agent maintains a long-running session across a two-day migration. On the second morning, the engineer knows that overnight CI has reformatted every file in two directories and that one dependency was upgraded.

They cannot easily enumerate which specific files the reformatting touched. The agent had read about thirty files the previous day.

The decisions made yesterday are still valid and were hard-won.

What is the most appropriate approach?

- | | |
|---|--|
| ☐ A Resume, and tell the agent that formatting changed but the logic did not | ☐ B Start a fresh session with a summary of yesterday's decisions and the current state |
| ☐ C Resume, and ask the agent to re-read all thirty files | ☐ D Resume and continue, since formatting changes do not affect logic |

◆ PAUSE AND DECIDE

What is the most appropriate approach?

- A

 Resume and explain
Reasonable for the formatting. It says nothing about the dependency upgrade, which can change behaviour.
- B

 Fresh session with a summary
Cannot enumerate what changed, and the decisions are worth carrying. That is exactly the summary case.
- C

 Re-read all thirty
Thirty stale versions plus thirty current ones, in one conversation. The worst option here.
- D

 Just continue
Treats a dependency upgrade as cosmetic. Formatting is harmless; the upgrade may not be.

THE SIGNAL

"Cannot easily enumerate what changed" is the tiebreaker from earlier in this module, quoted almost word for word.